

# Object-Oriented Programming Documentation

Official Reference Guide to Classes, Inheritance, and Polymorphism

---

## 1. Class Design & Encapsulation

Object-Oriented Programming (OOP) in Python models concepts as classes containing state (attributes) and behavior (methods). Encapsulation is achieved using private attribute conventions (`__attribute`) and property getters/setters.

```
class BankAccount:
    def __init__(self, owner: str, balance: float = 0.0):
        self.owner = owner
        self._balance = balance # Protected attribute

    @property
    def balance(self) -> float:
        return self._balance

    def deposit(self, amount: float):
        if amount <= 0:
            raise ValueError("Deposit amount must be positive")
        self._balance += amount

acc = BankAccount("Alice", 1000.0)
acc.deposit(500.0)
print(f"{acc.owner} Balance: ${acc.balance}")
```

## 2. Inheritance & Polymorphism

Inheritance allows subclasses to inherit properties and methods from parent classes. Method Resolution Order (MRO) determines method lookup order in multiple inheritance using C3 Linearization.

```
from abc import ABC, abstractmethod

class PaymentProcessor(ABC):
    @abstractmethod
    def process_payment(self, amount: float) -> bool:
        pass

class StripeProcessor(PaymentProcessor):
    def process_payment(self, amount: float) -> bool:
        print(f"Processing ${amount} via Stripe API...")
        return True

processor: PaymentProcessor = StripeProcessor()
processor.process_payment(250.0)
```